walrus-service 크레이트 분석 및 API 문서

개요

`walrus-service` 크레이트는 Walrus 분산 블롭 스토리지 시스템의 핵심 서비스 구현체입니다. 이 크레이트는 클라이언트 도구, 스토리지 노드 서버, 그리고 배포 유틸리티를 포함하여 Walrus 네트워크의 모든 주요 실행 가능한 바이너리를 제공합니다.

바이너리 구조

1. walrus (클라이언트)

파일: `bin/client.rs`

Walrus 블롭 스토어를 위한 명령줄 클라이언트입니다.

주요 기능:

- 블롭 저장 및 검색
- 메타데이터 관리
- 노드 상태 확인
- 데몬 모드 실행

사용 예제:

```
# 블롭 저장
walrus store file.txt
```

```
# 블롭 읽기
walrus blob <blob-id>

# 노드 상태 확인
walrus info

# 데몬 모드 실행
walrus daemon
```

2. walrus-node (스토리지 노드)

파일: `bin/node.rs`

Walrus 스토리지 노드를 관리하고 실행하는 바이너리입니다.

주요 명령어:

Setup (설정)

```
walrus-node setup \
--config-directory /opt/walrus/config \
--storage-path /opt/walrus/db \
--sui-network testnet \
--node-capacity 1TB \
--public-host node.example.com \
--name "MyWalrusNode"
```

Register (등록)

```
walrus-node  register \
--config-path /opt/walrus/config/walrus-node.yaml \
--force
```

Run (실행)

```
walrus-node  run \
--config-path /opt/walrus/config/walrus-node.yaml
```

Key Generation (키 생성)

```
# 프로토콜 키 생성
walrus-node  key-gen \
--key-type protocol \
--format  tagged \
--out protocol.key

# 네트워크 키 생성
walrus-node  key-gen \
--key-type network \
--format  pkcs8 \
--out network.key
```

3. walrus-deploy (배포 도구)

파일: `bin/deploy.rs`

Walrus 테스트베드 구성 및 시스템 컨트랙트 배포를 위한 도구입니다.

주요 명령어:

시스템 컨트랙트 배포

```
walrus-deploy deploy-system-contract \
--working-dir ./testbed \
--sui-network testnet \
--contract-dir ./contracts \
--host-addresses node1.example.com node2.example.com \
--n-shards 1000 \
--epoch-duration 1h
```

노드 등록

```
walrus-deploy register-nodes \
--client-config client_config.yaml \
--param-files node1-params.yaml node2-params.yaml \
--stake-amount 1000000000
```

HTTP API 상세 스펙

메타데이터 API

GET /v1/blobs/{blob_id}/metadata

블롭의 메타데이터를 조회합니다.

응답:

```
{
"blob_id": "M5YQinGO3RoRLaW_KbCfvXStVlWEqO5dGDe5cSiN07I",
"metadata": {
"unencoded_length": 1024,
"encoding_type": 1,
"merkle_root": "...",
"epochs": 5
}
}
```

PUT /v1/blobs/{blob_id}/metadata

블롭의 메타데이터를 저장합니다.

요청 본문: BCS 인코딩된 메타데이터

응답: `201 Created` 또는 `200 OK`

슬리버 API

GET /v1/blobs/{blob_id}/slivers/{sliver_pair_index}/{sliver_type}

특정 슬리버를 조회합니다.

매개변수:

- `sliver_type` : `primary` 또는 `secondary`
- `sliver_pair_index` : 슬리버 쌍 인덱스

응답: BCS 인코딩된 슬리버 데이터

PUT /v1/blobs/{blob_id}/slivers/{sliver_pair_index}/{sliver_type}

슬리버를 저장합니다.

요청 본문: BCS 인코딩된 슬리버 데이터

응답: `201 Created` 또는 `200 OK`

복구 심볼 API

GET /v1/blobs/{blob_id}/recoverySymbols/{symbol_id}

특정 복구 심볼을 조회합니다.

응답: BCS 인코딩된 복구 심볼 데이터

GET /v1/blobs/{blob_id}/recoverySymbols

복구 심볼 목록을 조회합니다.

쿼리 매개변수:

- `id` : 특정 심볼 ID 필터
- `targetSliver` : 대상 슬리버 인덱스
- `targetType` : 대상 슬리버 타입

상태 API

GET /v1/blobs/{blob_id}/status

블롭의 상태를 조회합니다.

응답:

```
{
"status": "Permanent",
"end_epoch": 100,
"is_certified": true,
"initial_certified_epoch": 50
}
```

GET /v1/health

노드의 건강성 정보를 조회합니다.

응답:

```
{
"uptime": "PT1H30M",
"epoch": 42,
"public_key": "...",
"node_status": "Active",
"shard_summary": {
"owned": 10,
"owned_shard_status": {
"ready": 8,
"in_transfer": 1,
"in_recovery": 1
```

```
}
}
}
```

설정 관리

노드 설정 구조

```
pub struct StorageNodeConfig {
// 기본 정보
pub name: String,
pub storage_path: PathBuf,
pub protocol_key_pair: PathOrInPlace<ProtocolKeyPair>,
pub network_key_pair: PathOrInPlace<NetworkKeyPair>,
// 네트워크 설정
pub public_host: String,
pub public_port: u16,
pub rest_api_address: SocketAddr,
pub metrics_address: SocketAddr,
// Sui 블록체인 연결
pub sui: Option<SuiConfig>,
// 스토리지 설정
pub voting_params: VotingParams,
```

```
pub commission_rate: u16,
// 고급 설정
pub blob_recovery: BlobRecoveryConfig,
pub shard_sync_config: ShardSyncConfig,
pub tls: TlsConfig,
}
```

설정 파일 예제

기본 설정:

```yaml
name: "walrus-node-1"
storage_path: /opt/walrus/db
protocol_key_pair:
path: /opt/walrus/config/protocol.key
network_key_pair:
path: /opt/walrus/config/network.key
public_host: node.example.com
public_port: 9185
rest_api_address: 0.0.0.0:9185
metrics_address: 127.0.0.1:9184

sui:
rpc: https://fullnode.testnet.sui.io:443
```

```yaml
system_object: 0x...
staking_object: 0x...
wallet_config: /opt/walrus/config/sui_config.yaml


voting_params:
storage_price: 100000
write_price: 20000
node_capacity: 250000000000


commission_rate: 6000
```

고급 설정:

```yaml
# 블롭 복구 설정
blob_recovery:
max_concurrent_blob_syncs: 100
max_concurrent_sliver_syncs: 2000
monitor_interval_secs: 60


# 샤드 동기화 설정
shard_sync_config:
sliver_count_per_sync_request: 1000
shard_sync_retry_min_backoff_secs: 60
```

```yaml
shard_sync_retry_max_backoff_secs: 600


# TLS 설정
tls:
disable_tls: false
certificate_path: /opt/walrus/config/certificate.pem


# HTTP/2 설정
rest_server:
http2_max_concurrent_streams: 1000
http2_adaptive_window: true
```

보안 및 인증

TLS 인증서 관리

자체 서명 인증서 생성:

```rust
fn create_self_signed_certificate(
key_pair: &NetworkKeyPair,
server_name: String,
) -> CertifiedKey {
let generated_server_name =
walrus_storage_node_client::server_name_from_public_key(key_pair.public());
```

```
let pkcs8_key_pair = to_pkcs8_key_pair(key_pair);


let mut params = CertificateParams::new(vec![
generated_server_name.clone(),
server_name
]).expect("valid subject_alt_names");
params.distinguished_name.push(DnType::CommonName, generated_server_name.clone()
let certificate = params.self_signed(&pkcs8_key_pair)
.expect("self-signing certificate must not fail");


CertifiedKey {
cert: certificate,
key_pair: pkcs8_key_pair,
}
}
```

요청 인증

디지털 서명 검증:

```
impl AuthenticatedSliver {
pub fn verify(&self, public_key: &PublicKey, merkle_root: &Hash) -> Result<(),
// 1. 머클 증명 검증
self.merkle_proof.verify(&self.sliver.hash(), merkle_root)?;
```

```
// 2. 디지털 서명 검증
public_key.verify(&self.sliver.to_bytes(), &self.signature)?;
Ok(())
}
}
```

모니터링 및 메트릭

Prometheus 메트릭

```
lazy_static! {
static ref BLOB_STORE_DURATION: HistogramVec = register_histogram_vec!(
"walrus_blob_store_duration_seconds",
"블롭 저장 소요 시간",
&["result"]
).unwrap();
static ref SLIVER_COUNT: GaugeVec = register_gauge_vec!(
"walrus_slivers_total",
"저장된 슬리버 총 개수",
&["node_id", "sliver_type"]
).unwrap();
static ref HTTP_REQUESTS_TOTAL: CounterVec = register_counter_vec!(
"walrus_http_requests_total",
```

```
"HTTP 요청 총 개수",
&["method", "endpoint", "status_code"]
).unwrap();
}
```

로깅 시스템

구조화된 로깅:

```
#[tracing::instrument(skip(self, blob_data))]
pub  async  fn  store_blob(&self, blob_data: &[u8], epochs: u32) -> Result<BlobI
let  span = tracing::info_span!("store_blob", blob_size = blob_data.len(), epoch
async  move {
tracing::info!("블롭 저장 시작");
let  encoded = self.encoder.encode(blob_data)
.map_err(|e| {
tracing::error!("인코딩 실패: {}", e);
e
})?;
tracing::info!("인코딩 완료, 슬리버 수: {}", encoded.slivers.len());
// 슬리버 배포 로직...
tracing::info!("블롭 저장 완료, ID: {}", blob_id);
Ok(blob_id)
}.instrument(span).await
```

```
}
```

성능 최적화

연결 풀링

```
pub struct ConnectionPool {
pools: HashMap<NodeId, Pool<HttpConnection>>,
max_connections: usize,
connection_timeout: Duration,
idle_timeout: Duration,
}

impl ConnectionPool {
pub async fn get_connection(&self, node_id: NodeId) -> Result<PooledConnectio
let pool = self.pools.get(&node_id)
.ok_or(NetworkError::NodeNotFound)?;
pool.get().await.map_err(NetworkError::from)
}
}
```

배치 처리

```rust
pub struct BatchRequest {
pub requests: Vec<SliverRequest>,
pub timeout: Duration,
pub max_concurrent: usize,
}


impl StorageNodeClient {
pub async fn batch_store_slivers(
&self,
batch: BatchRequest
) -> Result<Vec<StoreResult>, NetworkError> {
let semaphore = Arc::new(Semaphore::new(batch.max_concurrent));
let futures: Vec<_> = batch.requests
.into_iter()
.map(|req| {
let semaphore = semaphore.clone();
async move {
let _permit = semaphore.acquire().await?;
self.store_sliver_single(req).await
}
})
.collect();
```

```
timeout(batch.timeout, join_all(futures)).await?

}

}
```

배포 및 운영

Docker 배포

Dockerfile 예제:

```
FROM rust:1.70 as builder

WORKDIR /app

COPY . .

RUN cargo build --release --bin walrus-node


FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y ca-certificates

COPY --from=builder /app/target/release/walrus-node /usr/local/bin/

EXPOSE 9185 9184

CMD ["walrus-node", "run", "--config-path", "/etc/walrus/config.yaml"]
```

Kubernetes 배포

Deployment 예제:

```
apiVersion: apps/v1
```

```yaml
kind: Deployment
metadata:
name: walrus-node
spec:
replicas: 1
selector:
matchLabels:
app: walrus-node
template:
metadata:
labels:
app: walrus-node
spec:
containers:
- name: walrus-node
image: walrus-node:latest
ports:
- containerPort: 9185
- containerPort: 9184
volumeMounts:
- name: config
mountPath: /etc/walrus
- name: data
```

```yaml
mountPath: /opt/walrus/db

env:

- name: RUST_LOG

value: "walrus_service=info"

volumes:

- name: config

configMap:

name: walrus-config

- name: data

persistentVolumeClaim:

claimName: walrus-data
```

에러 처리

HTTP 상태 코드

성공 응답:

- `200 OK` : 성공적인 조회
- `201 Created` : 성공적인 생성
- `204 No Content` : 성공적인 삭제

클라이언트 에러:

- `400 Bad Request` : 잘못된 요청
- `401 Unauthorized` : 인증 실패

- `403 Forbidden` : 권한 없음
- `404 Not Found` : 리소스 없음
- `409 Conflict` : 충돌

서버 에러:

- `500 Internal Server Error` : 내부 서버 오류
- `503 Service Unavailable` : 서비스 사용 불가

에러 응답 형식

```
{
"error": {
"code": "BLOB_NOT_FOUND",
"message": "The requested blob was not found",
"details": {
"blob_id": "M5YQinGO3RoRLaW_KbCfvXStVlWEqO5dGDe5cSiN07I"
}
}
}
```

문제 해결

일반적인 문제

연결 문제:

```
# TLS 인증서 확인
openssl s_client -connect node.example.com:9185 -servername node.example.co

# 네트워크 연결 테스트
curl -k https://node.example.com:9185/v1/health
```

성능 문제:

```
# 메트릭 확인
curl http://127.0.0.1:9184/metrics

# 로그 분석
journalctl -u walrus-node -f
```

스토리지 문제:

```
# 디스크 사용량 확인
du -sh /opt/walrus/db

# 데이터베이스 상태 확인
walrus-node db-tool list-shards --db-path /opt/walrus/db
```

결론

`walrus-service` 크레이트는 Walrus 시스템의 실제 운영을 담당하는 핵심 구현체입니다. 클라이언트 도구부터 스토리지 노드 서버까지 모든 주요 바이너리를 제공하며, 강력한 HTTP API, 보안 메커니즘, 모니터링 기능을 통해 안정적인 분산 스토리지 서비스를 구현합니다.

주요 특징:

- **완전한 HTTP API**: RESTful 인터페이스를 통한 모든 기능 제공
- **강력한 보안**: TLS 암호화와 디지털 서명 기반 인증
- **확장 가능한 아키텍처**: 마이크로서비스 패턴과 모듈화 설계
- **운영 친화적**: 포괄적인 모니터링과 로깅 기능
- **배포 자동화**: Docker와 Kubernetes 지원

다음 단계: 05-sui-블록체인-통합에서 Sui 블록체인과의 통합 메커니즘을 살펴보세요.